



آن شب که دلی بود به میخانه نشستیم
آن توبه صدساله به پیمانہ شکستیم
از آتش دوزخ نهراسیم که آن شب
ما توبه شکستیم ولی دل نشکستیم

«مولانا»

۱. (۲۰ نمره) در تعریف و اندازه‌گیری هوشمندی، دو نگاه مبتنی بر Task و مبتنی بر Skill Acquisition Efficiency وجود دارد. این دو نگاه را با ذکر مثال، با یک دیگر مقایسه نمایید.

حل. در تعریف و اندازه‌گیری هوشمندی، دو رویکرد کلی مبتنی بر تسک (Task-centric) و بهینگی اکتساب مهارت (Skill Acquisition Efficiency) به شرح زیر وجود دارد:

(آ) نگاه مبتنی بر Task: در این دیدگاه، هوشمندی بر اساس توانایی سیستم در انجام موفقیت‌آمیز یک یا چند وظیفه‌ی مشخص سنجیده می‌شود. در واقع، عملکرد نهایی در یک حوزه‌ی معین، معیار اصلی هوشمند بودن تلقی می‌گردد.

- مثال: موفقیت یک سیستم در بازی شطرنج، رانندگی خودکار یا حل پازل‌های منطقی.
- محدودیت: ایراد اساسی این نگاه این است که میزان عملکرد به شدت تابع دانش پیشین (Prior Knowledge) و تجربه‌ی انباشته است. به عبارت دیگر، سیستم ممکن است صرفاً با تکیه بر حافظه و داده‌های زیاد در یک تسک خاص ماهر شود، بدون آنکه الزماً دارای هوشمندی تعمیم‌پذیر باشد.

(ب) نگاه مبتنی بر Skill Acquisition Efficiency: در این رویکرد، هوشمندی به معنای نرخ یا بهینگی یادگیری یک مهارت جدید با استفاده از حداقل داده تعریف می‌شود. در اینجا، تمرکز از «نتیجه‌ی نهایی» به «فرآیند یادگیری» تغییر می‌یابد.

- مثال: سرعت و کیفیت یادگیری یک بازی کاملاً جدید توسط ماشین، در حالی که تنها چند نمونه‌ی محدود از روند بازی را مشاهده کرده است.
- مزیت: در این نگاه، نقش تجربه و دانش پیشین به حداقل می‌رسد و آنچه اندازه‌گیری می‌شود، توانایی ذاتی سیستم برای تبدیل اطلاعات محدود به مهارت‌های کاربردی است. این معیار، به مفهوم واقعی هوشمندی و قدرت تعمیم نزدیک‌تر است.

۲. (۵۰ نمره) رهام که آخرین بار با خیانتی نابخشودنی ثابت کرد دیگر در بردگیم جایگاهی ندارد، حالا برای جبران شکست‌هایش ادعا کرده که می‌تواند با مفاهیم درس هوش مصنوعی، جای حافظ را در ادبیات بگیرد! او می‌خواهد فرآیند سرودن یک غزل زیبا را به عنوان یک مسئله جست‌وجو مدل‌سازی نماید. فرض کنید دایره واژگان زبان فارسی از حدود ۱۰۰۰۰ کلمه‌ی منحصر به فرد تشکیل شده است و یک غزل استاندارد را دنباله‌ای از حدود ۱۰۰ کلمه در نظر می‌گیریم.

(آ) (۲۰ نمره) این مسئله را به صورت رسمی برای رهام فرموله کنید. یعنی مجموعه‌ی استیت‌های مسئله، استیت اولیه، اکشن‌ها، هزینه‌ی هر اکشن، و استیت‌های هدف را مشخص نمایید.

(ب) (۱۰ نمره) رهام که کلا به کورکورانه عمل کردن عادت دارد (همانطور که در بازی عمل کرد)، می‌خواهد از الگوریتم‌های BFS یا DFS برای سرودن یک غزل استفاده کند. با تحلیل عددی به او توضیح دهید که چرا این رویکرد برای سرودن حتی یک بیت شعر هم خنده‌دار و غیرممکن است؟

(ج) (۱۰ نمره) برای اینکه الگوریتمی آگاهانه مانند A^* بتواند رهام را از خیره شدن به کاغذ خالی نجات دهد، به یک تابع هیوریستیک نیاز داریم. یک تابع هیوریستیک مناسب به او پیشنهاد دهید که بتواند مسیرهای اشتباه و بی‌معنی را زودتر رها کند.

(د) (۱۰ نمره) حافظ و رهام، هر دو دایره واژگان تقریباً مشابهی دارند و هر دو با مسئله‌ی جست‌وجوی یکسانی روبه‌رو هستند. اما حافظ می‌تواند در عرض چند دقیقه یا چند ساعت یک شاهکار ادبی بسراید، و رهام احتمالاً تا پایان ترم در لایه‌های ابتدایی درخت جست‌وجو گیر می‌کند. با آنچه از درس یاد گرفته‌اید، توضیح دهید تفاوت اصلی حافظ و رهام در چه چیزی می‌باشد؟

حل. دقت نمایید که این سوال یک پاسخ یکتا ندارد و به نوع مدل‌سازی و خلاقیت در تعریف اجزای مسئله وابسته است.

۱. فرموله‌سازی مسئله جست‌وجو: برای تبدیل سرودن غزل به یک مسئله‌ی جست‌وجو، اجزا را به صورت زیر تعریف می‌کنیم:

- مجموعه حالت‌ها (States): هر حالت مجموعه‌ای از کلمات پشت‌سرهم است که تا کنون سروده شده‌اند. یک حالت می‌تواند شامل از صفر کلمه (رشته تهی) تا حداکثر ۱۰۰ کلمه باشد.
- حالت اولیه (Initial State): یک لیست خالی از کلمات (نقطه شروع فرآیند سرودن).
- اعمال (Actions): انتخاب و اضافه کردن یک کلمه جدید از میان ۱۰۰۰۰ واژه‌ی موجود در دایره واژگان به انتهای رشته‌ی کلمات فعلی.
- هزینه اکشن‌ها (Action Cost): می‌توان آن را برای هر کلمه برابر با ۱ در نظر گرفت، یا براساس دشواری یافتن کلمه‌ای که با وزن و قافیه جور درمی‌آید، مقدار متفاوتی اختصاص داد.
- وضعیت‌های هدف (Goal States): حالت‌هایی که در آن دنباله شامل حدود ۱۰۰ کلمه باشد و تمامی قواعد عروضی (وزن)، قافیه، ردیف و پیوستگی معنایی در سطح یک «غزل زیبا» رعایت شده باشد.

۲. تحلیل عددی و نقد رویکردهای ناآگاهانه: در این مسئله، فاکتور انشعاب (Branching Factor) برابر با تعداد واژگان یعنی $b = 10,000$ است. عمق هدف نیز $d = 100$ در نظر گرفته شده است.

- تعداد حالت‌هایی که یک الگوریتم ناآگاهانه مانند BFS باید بررسی کند از مرتبه $O(b^d)$ یا $10^{10,000}$ است.

- این عدد (یک با ۴۰۰ صفر جلوی آن) به قدری بزرگ است که از تعداد اتم‌های جهان هستی نیز فراتر می‌رود. حتی برای سرودن یک بیت (حدود ۱۰ تا ۱۵ کلمه)، جست‌وجوی $10^{10,000}$ گره، قرن‌ها زمان بر روی قوی‌ترین ابرکامپیوترها نیاز دارد. بنابراین استفاده از این روش‌ها برای سرودن یک غزل یا بیت غزل عملاً غیرممکن است.

۳. پیشنهاد تابع هیوریستیک ($h(n)$): یک تابع هیوریستیک مناسب باید تخمینی از میزان «نامیدکننده بودن» مسیر فعلی ارائه دهد. برای این منظور، تابع زیر می‌تواند پیشنهاد می‌شود:

$$h(n) = w_1 \cdot \text{ProsodyError}(n) + w_2 \cdot \text{SemanticIncoherence}(n)$$

که در آن:

- ProsodyError : میزان انحراف کلمات انتخاب شده از وزن عروضی غزل را می‌سنجد. اگر کلمه‌ی جدید وزن را خراب کند، مقدار هیوریستیک بسیار بزرگ می‌شود تا آن شاخه قطع گردد.

• SemanticIncoherence: با استفاده از مدل‌های زبانی، میزان بی‌ارتباطی معنایی کلمه جدید با کلمات قبلی را می‌سنجد.

۴. تفاوت اصلی حافظ و یک فرد عادی: تفاوت اصلی در فضای جست‌وجوی موثر است. یک فرد عادی به دلیل نداشتن دانش تخصصی (خبرگی) در حوزه‌ی شعر و ذوق هنری، فضای جست‌وجو را کاملاً مسطح می‌بیند و مجبور است در هر گام میان تمام ۱۰,۰۰۰ کلمه جست‌وجو کند. اما فردی مثل حافظ دارای هیوریستیک‌های هنری بسیار قوی و توانایی هرس کردن ذهنی (Pruning) است. حافظ به واسطه نبوغ و تسلط ادبی، در هر مرحله ۹۹/۹٪ واژگان نامناسب را اصلاً وارد فضای جست‌وجوی ذهنی خود نمی‌کند. او به جای برخورد با ۱۰,۰۰۰ انتخاب، در هر مرحله تنها میان چند گزینه‌ی طلایی که همزمان وزن، معنا و ایهام را رعایت می‌کنند، جست‌وجو می‌کند.

۳. (۳۰ نمره) یک مسئله‌ی جست‌وجو را در نظر بگیرید. فرض کنید عدد ثابت و مثبت c را به هزینه‌ی تمام اکشن‌های مسئله اضافه می‌نماییم. در این صورت، آیا پاسخ نهایی الگوریتم UCS نسبت به حالت قبل، تغییر می‌کند؟ حال اگر همین عمل اضافه کردن یک ثابت مثبت را بر روی هیوریستیک‌های هر استیت مسئله انجام دهیم و به هزینه‌ها دست نزنیم، آیا خروجی الگوریتم A^* تغییر می‌کند؟ استدلال نمایید.

حل. تاثیر اضافه کردن مقدار ثابت مثبت به هزینه‌ها یا تابع هیوریستیک در الگوریتم‌های جست‌وجو به شرح زیر است:

۱. الگوریتم UCS (جست‌وجوی هزینه یکنواخت): در این حالت، پاسخ نهایی می‌تواند تغییر کند. فرض کنید هزینه‌ی یک مسیر P با تعداد k یال برابر با $G(P) = \sum_{i=1}^k cost(a_i)$ باشد. با اضافه کردن عدد ثابت c به هر اکشن، هزینه جدید برابر خواهد بود با:

$$G'(P) = \sum_{i=1}^k (cost(a_i) + c) = G(P) + k \cdot c$$

همان‌طور که مشاهده می‌شود، هزینه‌ی جدید به تعداد گام‌ها (k) بستگی دارد. این امر باعث می‌شود الگوریتم به سمت مسیریایی با تعداد گام‌های کمتر متمایل شود، حتی اگر در حالت اصلی مجموع هزینه‌ی آن‌ها بیشتر بوده باشد. به عنوان مثال، اگر مسیر اول یک گام با هزینه ۱۰ و مسیر دوم شامل پنج گام با هزینه کل ۹ باشد، ابتدا مسیر دوم انتخاب می‌شد. اما با اضافه کردن $c = ۱$ به هر اکشن، هزینه مسیر اول ۱۱ و هزینه مسیر دوم $۱۴ = ۵(۱) + ۹$ می‌شود و انتخاب الگوریتم به مسیر اول تغییر می‌یابد.

۲. الگوریتم A^* با اضافه کردن ثابت مثبت به هیوریستیک: در این حالت، پاسخ نهایی (مسیر انتخاب شده) تغییر نمی‌کند. الگوریتم A^* گره‌ها را بر اساس تابع $f(n) = g(n) + h(n)$ در صف اولویت قرار می‌دهد و از Fringe خارج می‌کند. اگر به تمام مقادیر هیوریستیک مقدار ثابت c اضافه شود، تابع ارزیابی جدید برابر خواهد بود با:

$$f'(n) = g(n) + (h(n) + c) = f(n) + c$$

از آنجا که مقدار c به ازای تمام اعضای موجود در لیست Fringe ثابت است، ترتیب نسبی گره‌ها در صف اولویت تغییر نخواهد کرد. یعنی اگر در حالت قبل $f(n_1) < f(n_2)$ برقرار بود، اکنون نیز $f(n_1) + c < f(n_2) + c$ برقرار است. در نتیجه، ترتیب بسط داده شدن گره‌ها (Expansion Order) دقیقاً مشابه حالت قبل باقی می‌ماند و الگوریتم همان مسیری را خروجی می‌دهد که در حالت اصلی انتخاب می‌کرد.